

Term Project – Proposal

Chess Club Tournament & Rating Manager

Dozent: Stephan Bökelmann · sboekelmann@ep1.rub.de · Technische Hochschule Georg Agricola, Bochum

Author: Yahya El Idrissi

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekelmann@ep1.rub.de

Target size: about 6 pages

1 Application domain and motivation

The project is a **Chess Club Tournament & Rating Manager**: a small, self-hosted system that lets a chess club run Swiss-system tournaments, record game results, maintain Elo ratings over time and answer performance questions across a player's whole game history. Small clubs today typically juggle a spreadsheet for the member list, a second spreadsheet per tournament for the pairings and results, and no persistent record of ratings or openings played – which makes any resembling question (“how does player X score with Black in the Sicilian?”) practically unanswerable. Consolidating this into a single relational store solves that.

Two concrete usage scenarios illustrate the intended use:

- **Tournament director, Friday evening.** The director opens a new four-round Swiss tournament, registers twelve players from the existing member list, records the results of round 1 through the frontend and prints the standings after each round. Elo ratings update automatically when the tournament is closed.
- **Club member, at home.** A player opens the frontend, points it at the club server and looks up their personal statistics: overall win/draw/loss counts, score by colour, score per opening and Elo progression over the last twelve months.

2 Data model – ER sketch

Figure 1 shows the data model. The central entity types are **player** (name, birth year, current Elo), **tournament** (name, start date, number of rounds, status), **round** (round number within a tournament), **game** (linking two players and one opening within a round, plus the result), **opening** (ECO code, name) and **rating_history** (Elo value of a player at a point in time). The mandatory **N:M relationship** is *player* ↔ *tournament*: a player takes part in many tournaments over time, and a tournament has many players. It is resolved by the associative entity **registration**, which carries the seed and the final rank. Every **game** connects two players (white and black), belongs to exactly one round, and uses exactly one opening. The relationship *tournament* → *round* → *game* is a strict 1:N chain, and *player* → *rating_history* is 1:N as well.

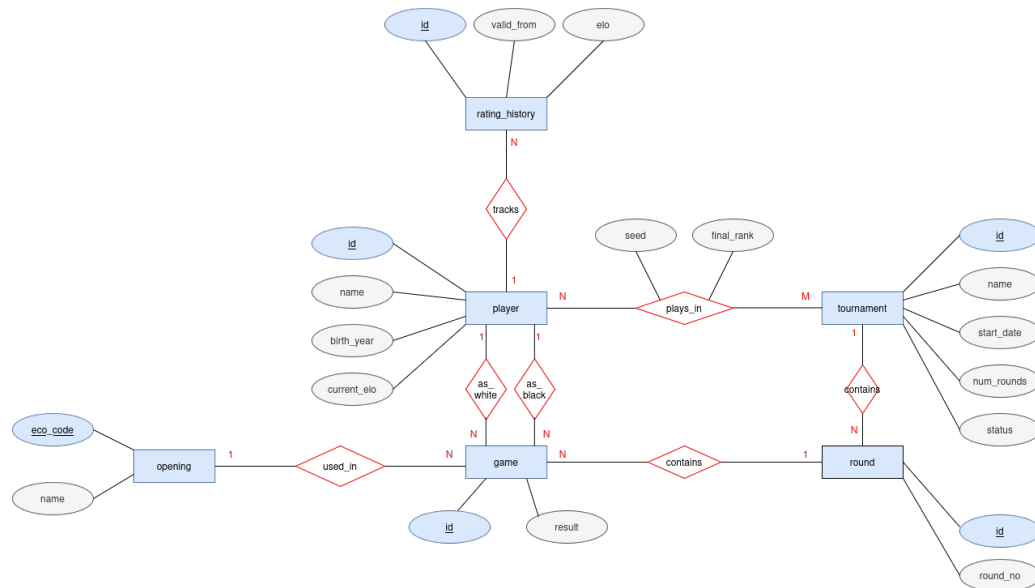


Abbildung 1: ER sketch of the chess-club data model: entity types, key attributes, relationships and cardinalities. The central N:M relationship is $player \leftrightarrow tournament$, realised through the associative entity **registration**.

3 From model to schema

Applying the ER-to-relational mapping rules to Figure 1 yields a schema of seven tables: each strong entity becomes a table, the N:M relationship **plays_in** becomes its own table **registration** carrying the relationship attributes **seed** and **final_rank**, and the remaining 1:N relationships become foreign keys on the N-side.

- **player**(id, name, birth_year, current_elo)
- **opening**(eco_code, name)
- **tournament**(id, name, start_date, num_rounds, status)
- **registration**(player_id, tournament_id, seed, final_rank)
- **round**(id, tournament_id, round_no), UNIQUE(tournament_id, round_no)
- **game**(id, round_id, white_id, black_id, eco_code, result),
CHECK(result IN ('1-0', '0-1', '1/2-1/2')),
CHECK(white_id $\neq$ black_id)
- **rating_history**(id, player_id, valid_from, elo)

Foreign keys: **registration.player_id** $\rightarrow$ **player**, **registration.tournament_id** $\rightarrow$ **tournament**, **round.tournament_id** $\rightarrow$ **tournament**, **game.round_id** $\rightarrow$ **round**, **game.white_id**, **game.black_id** $\rightarrow$ **player**, **game.eco_code** $\rightarrow$ **opening**, **rating_history.player_id** $\rightarrow$ **player**.

Normalisation. Every non-key attribute depends on the whole primary key and only on it: opening names live in **opening**, not repeated in **game**; a player's current Elo is stored once in **player** and its history is factored into **rating_history**. There are no transitive dependencies, so the schema is in **3NF**.

4 API design

The FastAPI layer exposes read endpoints for public statistics and standings and write endpoints (protected by X-API-Key) for anything that mutates club data.

Method	Path	Purpose
GET	/players	list of players with total games played, computed as <code>COUNT(*)</code> over <code>game</code> where the player appears as <code>white_id</code> or <code>black_id</code> (<i>JOIN + aggregation</i>)
GET	/players/{id}/stats	win/draw/loss and average Elo delta grouped by <code>opening.name</code> (<i>JOIN game $\bowtie$ opening + aggregation</i>)
GET	/tournaments	list of all tournaments
GET	/tournaments/{id}/standings	points per player + Buchholz tiebreak (<i>JOIN across 4 tables + aggregation</i>)
POST	/players	create a new player (<i>X-API-Key</i>)
POST	/tournaments	create a new tournament (<i>X-API-Key</i>)
POST	/tournaments/{id}/registrations	register a player (<i>X-API-Key</i>)
POST	/games	record a game result, triggers Elo update (<i>X-API-Key</i>)

The signature endpoint is `GET /tournaments/{id}/standings`: it JOINS `registration` to `player` on `player_id`, and `game` to `round` on `round_id` (filtered by `round.tournament_id`), then groups by player to compute `SUM(points)` plus the Buchholz sum of the opponents' scores. All four POST endpoints require the X-API-Key header.

5 Frontend prototype (hand sketches)

The frontend is a **Python/tkinter** desktop application, packaged as a `.deb` installer as required, and asking for the API URL and X-API-Key in a connection dialog at startup. tkinter is chosen because it ships with the Python standard library and packages cleanly into a Debian archive.

Figures 2–5 show the four screens of the prototype. Figure 2 shows the connection dialog that collects the API URL and X-API-Key at startup and stores them in the running session. Figure 3 shows the standings screen, the main list view of the application, which reads from `GET /tournaments/{id}/standings` and refreshes on demand. Figure 4 shows the record-game form used by the tournament director to enter a result; it calls `POST /games` with the X-API-Key header. Figure 5 shows the player-stats screen, which reads `GET /players/{id}/stats` and renders the win/draw/loss breakdown per opening.

6 Operation with Docker Compose

The `docker-compose.yml` defines two services: `postgres` (official `postgres:16` image, with the schema and seed data mounted as an init script under `/docker-entrypoint-initdb.d`) and `api` (the FastAPI application, built from a local `Dockerfile`). Credentials – `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` and `API_KEY` – live in a `.env` file that is `.gitignore`-d. Deployment on the lecture server is one step: clone the repository, place `.env` beside the compose file and run `docker compose up -d`. The frontend `.deb` is installed separately on the same server via `sudo dpkg -i chess-club-fe.deb` and connects to the API over HTTP.

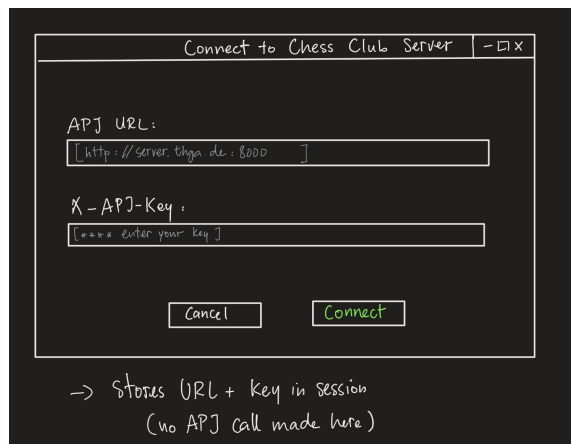


Abbildung 2: Startup connection dialog: API URL + X-API-Key.

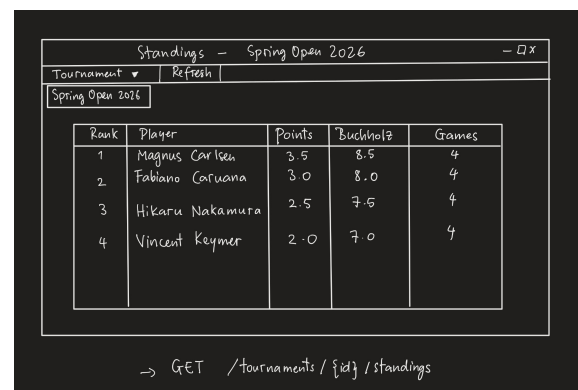


Abbildung 3: Standings screen: table of players, calls GET /tournaments/{id}/standings.

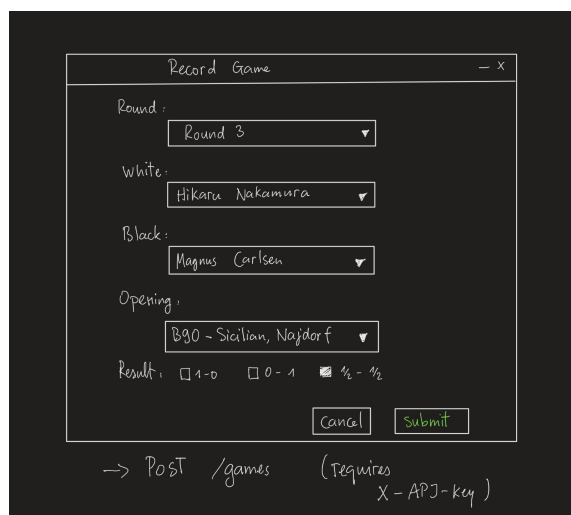


Abbildung 4: Record-game form: white, black, opening, result – calls POST /games.

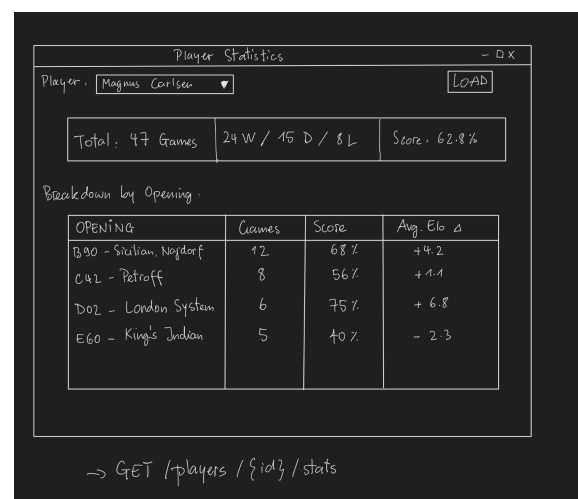


Abbildung 5: Player-stats screen: calls GET /players/{id}/stats.

7 Scope, milestones and risks

Scope. 7 tables, 8 endpoints (4 read, 4 write), 4 tkinter screens plus the connection dialog.

Milestones and schedule.

Week	Milestone	Est. hours
1	Schema, DDL, seed data (players + one closed tournament) ready in PostgreSQL	8
2-3	FastAPI: read + write endpoints, X-API-Key dependency, standings query with JOIN + aggregation	12
4-5	tkinter frontend (4 screens + connection dialog); .deb builds and installs on the lecture server	10
6	pytest test suite, documentation (LaTeX repo with CI), 8-10 min video	10

Risks. (1) The Elo update on `POST /games` is a small business rule with real edge cases (both players' ratings must move symmetrically, and recording a game against an unregistered player must be refused). Getting this subtly wrong is the most likely way to fail correctness tests. (2) Packaging the tkinter frontend as a working `.deb` on the lecture server is a build-system risk unrelated to the database itself.

8 Acceptance criteria and testing

The system counts as done and correct when the criteria below hold on the lecture server, verified by an automated test suite.

Acceptance criteria.

- A `POST /games` with `white_id = black_id` is rejected with HTTP 400 (CHECK constraint).
- A `POST /games` whose `result` is not in `{ '1-0', '0-1', '1/2-1/2' }` is rejected with HTTP 400.
- Registering the same player twice for the same tournament is rejected with HTTP 409 (primary key on `registration`).
- Recording a game for a player who is not registered in that tournament is rejected with HTTP 409.
- `GET /tournaments/{id}/standings` returns the correct `SUM(points)` per player and Buchholz tiebreak (JOIN + aggregation).
- Every `POST` endpoint without a valid `X-API-Key` returns HTTP 401.

Test plan.

- **Database:** PK on all tables, composite PK on `registration`, FKs on `round`, `game`, `registration`, `rating_history`, UNIQUE on `(tournament_id, round_no)`, CHECK on `game.result` and on `white_id ≠ black_id`, NOT NULL on all foreign keys.
- **API:** `pytest` with FastAPI's `TestClient` against a throw-away PostgreSQL container. Happy paths for all four `GET` endpoints and all four `POST` endpoints, plus 4xx error cases for each acceptance criterion above.
- **Business rule (risk 1).** Dedicated test that records a game for a player *not* registered in the tournament and asserts HTTP 409; a second test records a legal game and asserts that both players' Elo values change symmetrically and that a `rating_history` row is written for each.

Effort budget: the whole term project (system, documentation and video) fits into roughly **40 h of work** over at most **2 months**. The milestone table above sums to exactly that budget.